



SafeWebCore

Aligning Zero-Configuration Security Headers with OWASP Best Practices

Author: M. Posseth (MPCoreDeveloper)

Date: March 2026

About the Author

M. Posseth is a passionate .NET developer focused on building secure, high-performance applications. SafeWebCore is an open-source project created to make modern security headers easily accessible to the .NET community.

Project Links

GitHub Repository <https://github.com/MPCoreDeveloper/SafeWebCore>

NuGet Package <https://www.nuget.org/packages/SafeWebCore>

Installation

C#

```
dotnet add package SafeWebCore
```

SafeWebCore is a lightweight ASP.NET Core middleware library that helps developers add robust security headers with minimal effort, achieving an A+ rating on securityheaders.com out of the box.



SafeWebCore

Foreword

In the ever-evolving landscape of web application security, implementing robust HTTP security headers remains one of the most effective and low-effort ways to reduce attack surface and achieve strong protection against common web vulnerabilities.

SafeWebCore is a lightweight, high-performance middleware library for ASP.NET Core (.NET 10) that enables developers to add modern security headers with **zero configuration** — delivering an **A+ rating** on securityheaders.com (powered by Snyk) out of the box through its `AddNetSecureHeadersStrictAPlus()` preset.

This article explores how SafeWebCore relates to key OWASP initiatives, including the OWASP Top 10, the HTTP Security Headers Cheat Sheet, and the OWASP Secure Headers Project (OSHP). It details which risks are mitigated, how the defaults align with industry best practices, and the deliberate trade-offs in the strict A+ preset.



SafeWebCore

Section 1: OWASP Risks Mitigated by SafeWebCore

SafeWebCore operates at the HTTP response layer as a defense-in-depth measure. It directly helps mitigate several risks from the **OWASP Top 10** (2021 and 2025 editions):

- **A03: Injection** (including Cross-Site Scripting – XSS)

Through a strict nonce-based **Content Security Policy (CSP)** with `strict-dynamic`, Trusted Types support, and the absence of `unsafe-inline`. This significantly reduces the risk of script injection and DOM-based XSS.

- **A05 / A02: Security Misconfiguration**

Automatic removal of the Server header, sensible defaults for all security headers, and prevention of information leakage.

- **A06: Vulnerable and Outdated Components & Sensitive Data Exposure**

Strong **HSTS** (HTTP Strict Transport Security), `X-Content-Type-Options: nosniff`, and Cross-Origin policies (COEP, COOP, CORP) that help prevent downgrade attacks, MIME sniffing, and data exfiltration.

- **Clickjacking (UI Redressing)**

`X-Frame-Options: DENY` combined with `CSP frame-ancestors 'none'`.

- **Cross-Origin Attacks & Side-Channel Attacks**

Full Cross-Origin Isolation via `Cross-Origin-Embedder-Policy: require-corp`, `Cross-Origin-Opener-Policy: same-origin`, and `Cross-Origin-Resource-Policy: same-origin`.

- **Feature Policy Abuse**

A restrictive **Permissions-Policy** that disables all 29 browser features by default.



SafeWebCore

These protections also contribute to higher levels of the **OWASP Application Security Verification Standard (ASVS)**, particularly in sections covering header hygiene and CSP enforcement.

Section 2: Alignment with OWASP Best Practices

The **Strict A+ preset** in SafeWebCore closely follows recommendations from:

- OWASP HTTP Security Headers Cheat Sheet
- OWASP Secure Headers Project (OSHP) best practices

Key alignments include:

- **Strict-Transport-Security:** `max-age=63072000; includeSubDomains; preload` — matches OWASP's recommended 2-year maximum with preload support.
- **X-Frame-Options** + CSP `frame-ancestors 'none'` — the modern, preferred approach.
- **X-Content-Type-Options:** `nosniff` — direct OWASP recommendation.
- **Server** header removal — eliminates fingerprinting.
- **Cross-Origin policies** — aligned with modern browser isolation guidance.
- **Content-Security-Policy** — nonce-based, Level 3 compliant, with Level 4 readiness (Trusted Types).

Small intentional deviations make the preset stricter than the minimum:

- **Referrer-Policy:** `no-referrer` (instead of `strict-origin-when-cross-origin`)
- **Permissions-Policy:** denies **all** features (maximum lockdown)



SafeWebCore

Section 3: Trade-offs and Assumptions in the A+ Preset

The Strict A+ preset prioritizes **maximum security** with zero configuration. This comes with trade-offs:

- **Strict CSP** may break third-party integrations (CDNs, widgets, OAuth flows, analytics) unless explicitly allowed via the fluent CSP builder.
- Assumes a modern application that uses nonces for inline scripts/styles and does not rely on legacy browser behaviors.
- Ideal for internal tools, APIs, or tightly controlled single-page applications. For public sites with many external dependencies, use the customizable `AddNetSecureHeaders()` method and relax policies selectively.

The library provides excellent developer experience: a type-safe fluent API, per-request nonce generation (`HttpContext.GetCspNonce()`), built-in CSP violation reporting, and full extensibility.

Section 4: Validation Against Scanners

SafeWebCore's Strict A+ preset is designed and tested to achieve an **A+ score** on securityheaders.com. It also performs well against Google's CSP Evaluator.

Regarding the **OWASP Secure Headers Project validator** (the Venom-based OSHP validator):

As a very new library, formal testing against the OSHP validator is planned shortly. Given the close alignment with the OWASP Cheat Sheet and Secure Headers best practices, we expect strong or full compliance. Results and any minor adjustments will be documented in the project's docs.



SafeWebCore

Conclusion

SafeWebCore makes enterprise-grade security headers accessible to .NET developers without the usual configuration burden. By closely aligning with OWASP guidelines while offering both a strict out-of-the-box preset and powerful customization options, it helps teams ship more secure applications faster.

Try it today:

```
builder.Services.AddNetSecureHeadersStrictAPlus();  
app.UseNetSecureHeaders();
```

Repository: <https://github.com/MPCoreDeveloper/SafeWebCore>

Documentation: /docs/ (getting-started, security-headers, presets, csp-configuration)

We welcome contributions, feedback, and test results, especially from the OWASP community.

References

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- OWASP HTTP Security Headers Cheat Sheet: <https://cheatsheetseries.owasp.org/cheatsheets/HTTP-Headers-Cheat-Sheet.html>
- OWASP Secure Headers Project (OSH): <https://owasp.org/www-project-secure-headers/>
- OWASP Secure Headers Validator: <https://github.com/oshp/oshp-validator>
- securityheaders.com (Snyk): <https://securityheaders.com/>
- SafeWebCore GitHub Repository: <https://github.com/MPCoreDeveloper/SafeWebCore>